



Module 2 Final Project

Course:ALY6110-Data Management and Big Data

Academic Term-Fall 2025

Instructor: Professor Mary Donhoffner

Group 2

Princess Chishato

Muhammad Waqas Azam

Snovy Sunil Dbritto

Junhui Hu

Ruman Liu

Shyam Sundar Nandakumar

Contents

Introduction	3
Data Import	3
Data Cleaning	4
Summary Statistics	5
User Behavior Aggregation	7
Visual Analysis	9
Conclusion	11
Reference	13

Introduction

In order to improve the understanding of consumer purchasing behavior, we conduct exploratory data analysis on Instacart data, which lays the foundation for feature selection. Focusing on order patterns and the purchasing time of different product categories, we also look at the number of active users. In the final EDA, we made a number of changes in terms of the presentation and content, including an increased focus on the “why” of each step and ensuring clarity on how variables are being handled, but also correcting the definition of missing values to avoid jumping to conclusions about customer loyalty.

Data Import

We initially considered analyzing each data table separately (orders, products, aisles, and departments) to reduce computation. However, we later realized that key behavioral relationships, such as how often users repurchase products from specific departments which could only be captured if all tables were combined into one comprehensive dataset.

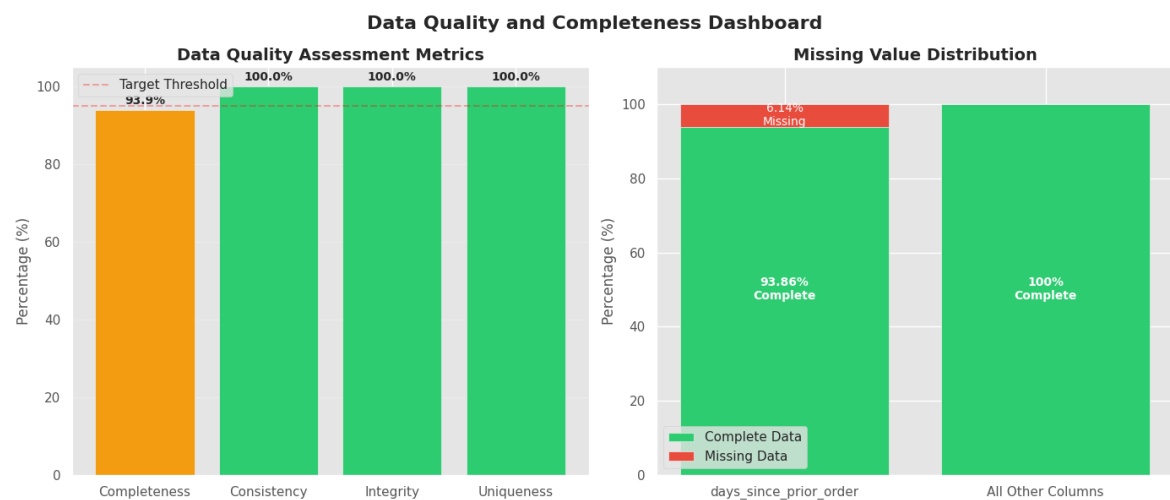
The Instacart data set is a set of online grocery orders made through Instacart's e-commerce site. Departments and aisles are two different levels of product category granularity used in Instacart's online catalog. Data dictionary shows below:

	A	B	C	D	E
1	Variable Name	Description	Type	Source Table	Example Value
2	order_id	Unique identifier for each customer order.	Integer	orders / order_products	2539329
3	user_id	Unique identifier for each customer.	Integer	orders	112108
4	eval_set	Specifies whether the order belongs to the train, test, or prior dataset.	String (categorical)	orders	prior
5	order_number	Sequential order number for each user (e.g., 1 = first order).	Integer	orders	4
6	order_dow	Day of the week the order was placed (0 = Sunday, 6 = Saturday).	Integer	orders	5
7	order_hour_of_day	Hour of the day when the order was placed (0-23).	Integer	orders	14
8	days_since_prior_order	Number of days since the user's previous order. Missing for first order.	Float	orders	9
9	product_id	Unique identifier for each product.	Integer	order_products / products	24852
10	add_to_cart_order	Sequence in which products were added to the cart during checkout.	Integer	order_products	1
11	reordered	1 if the product was previously ordered by the same user, 0 otherwise.	Integer (binary)	order_products	1
12	product_name	Name of the product.	String	products	"Organic Avocado"
13	aisle_id	Identifier of the aisle where the product is located.	Integer	products / aisles	83
14	department_id	Identifier of the department where the product belongs.	Integer	products / departments	4
15	aisle	Descriptive name of the aisle.	String	aisles	"fresh vegetables"
16	department	Descriptive name of the department.	String	departments	"produce"

We used left joins on product and order IDs to preserve all transaction records. After merging, the dataset contained over 33 million rows, representing historical transactions from 206,000 unique users. By the way, since one of the datasets is too large, we had to upload it to the catalog and read it using `spark.read.table()`. Databricks is quite a unique platform in this sense.

Data Cleaning

We found that the only variable with missing values was `days_since_prior_order` (6.14%).



When we first filled the missing days with 0, it was mentioned by the instructor that could be used to signify “daily shoppers”. We changed it to -1 to mean we don't know what the value is for unknown/new customers so that our clustering models down the line will treat these as separate/not-loyal customers.

```

07:19 PM (5s)
# missing value
missing = full_data.isnull().sum().sort_values(ascending=False)
missing = missing[missing > 0]
missing_percent = (missing / len(full_data) * 100).round(2)
missing_df = pd.DataFrame({'Missing Values': missing, 'Percent': missing_percent})
print(missing_df)

missing_df: pandas.core.frame.DataFrame = [Missing Values: int64, Percent: float64]
Missing Values  Percent
days_since_prior_order    2078068    6.14

07:19 PM (3s)
full_data['days_since_prior_order'].fillna(-1, inplace=True)
full_data['aisle'].fillna('Unknown', inplace=True)
full_data['department'].fillna('Unknown', inplace=True)

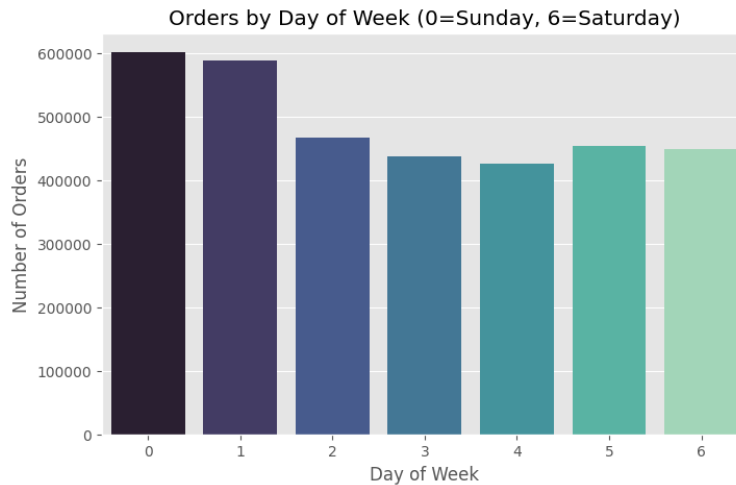
```

Summary Statistics

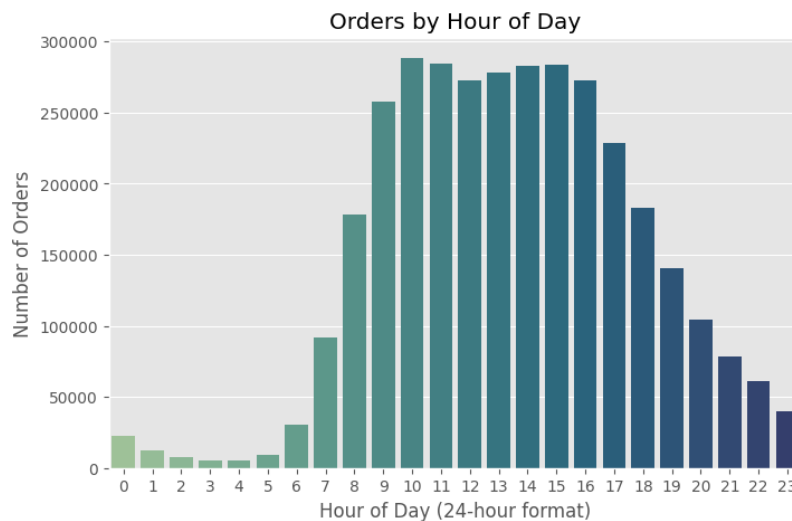
In order to have an exploratory understanding of the customers' purchasing behavior, we performed a descriptive statistical analysis of the few numerical variables present in the data that may help us to identify behavioral trends. The variables we selected were `order_dow`, `order_hour_of_day`, `days_since_prior_order` and `add_to_cart_order`, that together describe the time of purchase on Instacart.

	count	mean	std	min	25%	50%	75%	max
order_dow	33819106.0	2.737285	2.093296	0.0	1.0	3.0	5.0	6.0
order_hour_of_day	33819106.0	13.431226	4.246149	0.0	10.0	13.0	16.0	23.0
days_since_prior_order	33819106.0	10.604418	9.156268	-1.0	4.0	7.0	15.0	30.0
add_to_cart_order	33819106.0	8.367738	7.139540	1.0	3.0	6.0	11.0	146.0

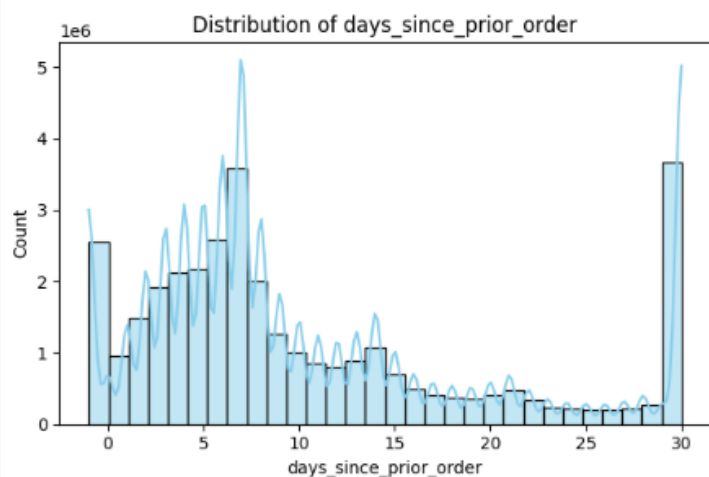
The variable `order_dow`, which indicates the day of the week when an order was placed, shows that most purchases occur between Tuesday and Thursday. The mean value of approximately 2.7 confirms a mid-week shopping preference among customers, which may suggest that many users restock essential items after the weekend.



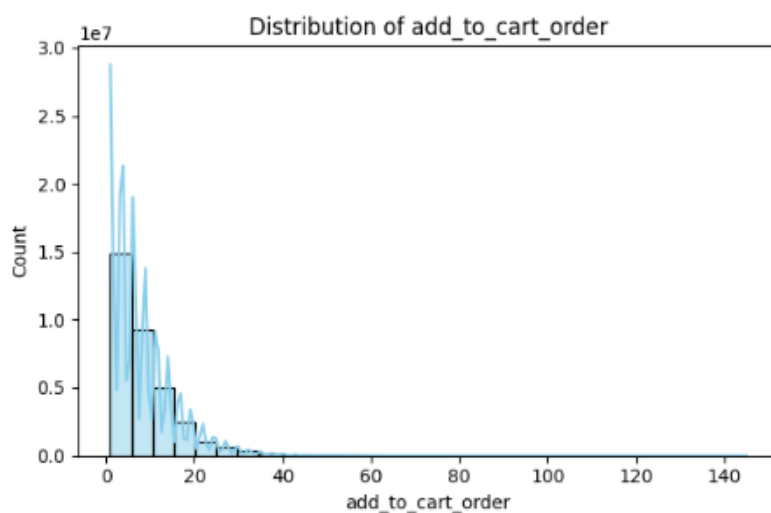
The `order_hour_of_day` variable also points out that the shopping is carried out generally in early afternoon, with an average hour of 13.4. This is also coherent with users ordering in online grocery stores during their work breaks.



`days_since_prior_order` records the number of days elapsed between orders. The average order interval is close to 10 days. The median interval is around 7 days, and the maximum is 30. This variable is right skewed. In other words, most users are weekly or biweekly shoppers and there are some longer-term infrequent buyers who place an order once per month or less.



Finally, the variable `add_to_cart_order`, which gives the order in which products are added to the user's cart, has a highly right-skewed distribution. While most products are added towards the beginning of the checkout process, a few large orders include more than one hundred products, corresponding to large "stock-up" baskets for high-value or family-oriented users.



User Behavior Aggregation

Then we turned to user-level feature aggregation to construct behavioral indicators that can later support segmentation and dimensionality reduction. These features summarize each customer's shopping frequency, timing patterns, and engagement with reorders.

More specifically, we calculated the total number of orders per user and the average days between orders. We also calculated the most frequent day and hour of shopping. Measures like average and maximum cart size, which indicates how many products a user purchases in a single transaction, were derived as well.

The summary statistics show that there is interesting variation across users. The minimum days between reorders and the maximum days between reorders show two extremes. The reorders have happened for some customers every few days, while for others it has been a year or longer. This suggests users who have regular grocery needs and make Instacart their default grocery shop and other users who have one-time or sporadic needs.

	user_id	total_orders	avg_days_between_orders
0	1	11	17.757143
1	2	15	16.915929
2	3	12	10.068182
3	4	5	11.722222
4	5	5	9.130435
...	...	...	...
206204	206205	4	9.470588
206205	206206	67	3.971930
206206	206207	16	13.170404
206207	206208	49	7.292467
206208	206209	14	18.824818

206209 rows × 3 columns

	user_id	avg_order_hour	most_common_order_dow
0	1	10.142857	4
1	2	10.517699	1
2	3	16.352273	0
3	4	13.111111	4
4	5	14.804348	0
...	...	...	...
206204	206205	14.509804	1
206205	206206	16.796491	0
206206	206207	13.130045	1
206207	206208	13.968981	2
206208	206209	12.985401	1

206209 rows × 3 columns

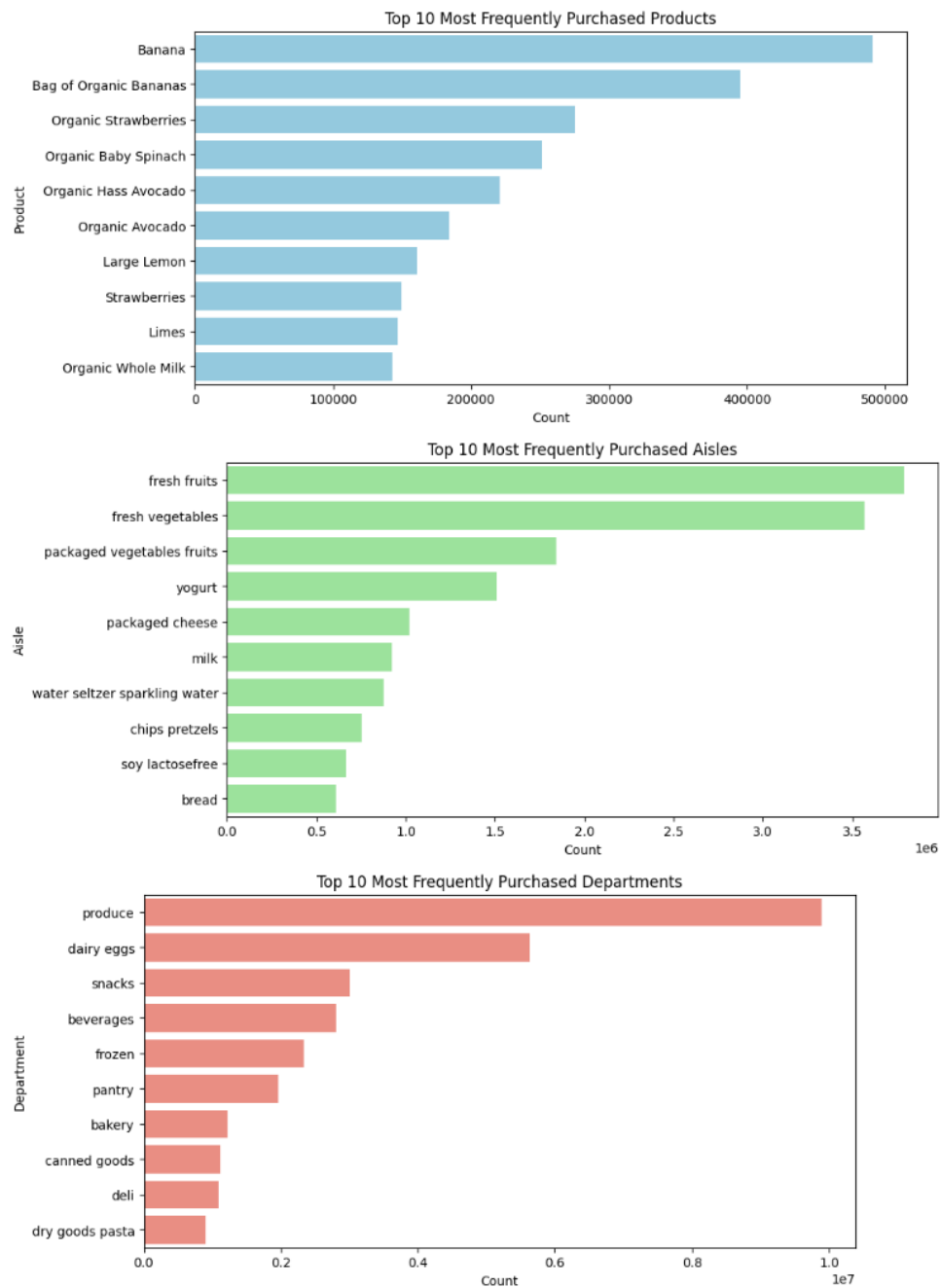
user_id reorder_rate			user_id avg_cart_size max_cart_size			
0	1	0.728571	0	1	6.363636	11
1	2	0.464602	1	2	15.066667	31
2	3	0.625000	2	3	7.333333	11
3	4	0.055556	3	4	3.600000	7
4	5	0.391304	4	5	9.200000	12
...	...	...	...	...	...	...
206204	206205	0.274510	206204	206205	12.750000	19
206205	206206	0.473684	206205	206206	4.253731	24
206206	206207	0.587444	206206	206207	13.937500	25
206207	206208	0.707533	206207	206208	13.816327	28
206208	206209	0.474453	206208	206209	9.785714	20

206209 rows × 2 columns 206209 rows × 3 columns

Visual Analysis

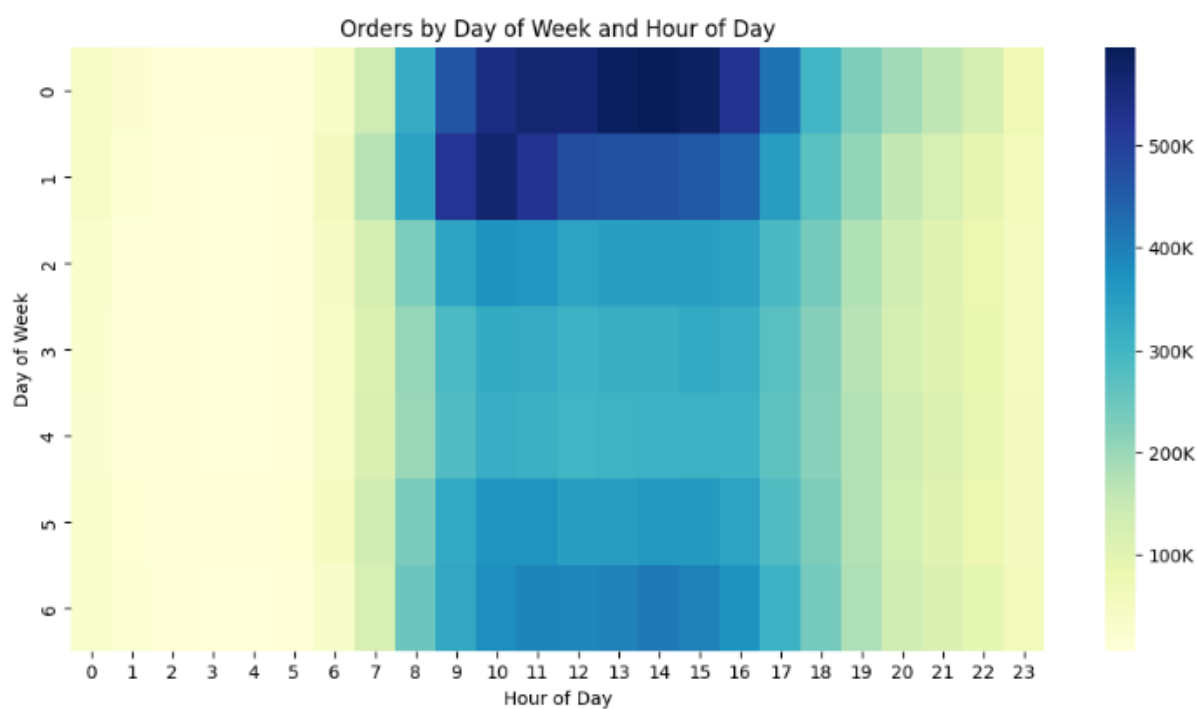
The first analysis focused on identifying the most frequently purchased products, aisles, and departments. The top ten product chart clearly shows that fresh produce dominates Instacart orders, with bananas, organic strawberries, and avocados ranking among the highest purchased items.

The visual analysis was intended to answer the questions of how Instacart users shop by category and by time. The graphs of the top products, aisles, and departments convey that majority of Instacart orders are coming from the fresh produce items, such as bananas, strawberries, avocados. The department graph shows higher level categories such as Produce or Snacks. The aisle graph further breaks down each department into sub-categories, such as fresh fruits or fresh vegetables.



A heatmap(LikeGeeks, 2023) of orders by day and hour shows that shopping peaks between 9 a.m. and 4 p.m., mostly on weekends, consistent with typical online grocery habits. The color bar was simplified to thousands (K) to improve readability. A treemap (Laserson,U, 2016) of departments confirms that product variety is concentrated in Produce, Pantry, and Snacks, indicating users' preference for essential grocery items.

Department Product Tree Map



Conclusion

This EDA is intended to explore the patterns and structure inherent in Instacart's consumer data, not to provide predictive insights. Every single step, from merging all

tables into a single collective dataset to looking into temporal, categorical, and behavioral characteristics, was done in order to explore how users categorize themselves naturally into behavioral groups. The emphasis on visualization and descriptive statistics inherently acknowledges the unsupervised nature of the study design: we are describing relationships that exist between variables rather than measuring predictive weighting on what determines importance.

Correspondingly, the omission of the traditional next step of looking at "features" to assess the importance of variables, is purposeful, given that "feature importance" is applied to supervised learning, in which a target variable defines the model. In this context, our approach is to explore the structure and variability of user behavior, including how often customers shop, the categories predominantly in their carts, and the relationship between time and frequency. By this straightforward focus on the underlying relationships rather than predictions, our EDA also provides a structure for clustering, and aims to unpack hidden behavioral patterns rather than predict what these purchases will be in the future.

Reference

Kaggle. (2017). Instacart market basket analysis dataset. Kaggle.

<https://www.kaggle.com/datasets/psparks/instacart-market-basket-analysis>

Laserson, U. (2016). Squarify: Treemap implementation for Python (matplotlib).

GitHub. <https://github.com/laserson/squarify>

Python Graph Gallery. (2024). Treemap charts with matplotlib and squarify.

<https://python-graph-gallery.com/treemap>

LikeGeeks. (2023, August 10). Customize Seaborn heatmap colorbar.

<https://likegeeks.com/seaborn-heatmap-colorbar>

Scikit-learn developers. (2024). Clustering — scikit-learn 0.24 user guide. Scikit-

learn. <https://scikit-learn.org/stable/modules/clustering.html>